



UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS CRATEÚS
SEMESTRE 2026.1

Resolução do Problema do Caixeiro Viajante utilizando Heurísticas e
Busca Local
TRABALHO FINAL

ALUNO

Luiz Gustavo Macedo Magalhães 556217

PROFESSOR

Rafael Martins Barros

Conteúdo

1	Introdução	3
2	Metodologia e Heurísticas	3
3	Resultados Experimentais	4
4	Uso de Ferramentas de IA	6
5	Conclusão	6
6	Referências Bibliográficas	6

1 Introdução

O Problema do Caixeiro Viajante (PCV, ou TSP em inglês) é um desafio clássico na área de otimização combinatória. Basicamente, dado um conjunto de cidades e as distâncias entre elas, o objetivo é achar a rota mais curta que passe por todas as cidades exatamente uma vez e depois retorne para a origem (um ciclo hamiltoniano de custo mínimo).

Como visto nas aulas e na bibliografia base (Szwarcfiter, 2018), trata-se de um problema NP-Difícil. Isso implica que tentar encontrar a resposta ótima por força bruta torna-se inviável computacionalmente conforme a dimensão do problema cresce. O foco deste trabalho prático foi desenvolver algoritmos heurísticos capazes de obter resultados de alta qualidade em instâncias simétricas reais da TSPLIB, respeitando o limite de tempo computacional.

2 Metodologia e Heurísticas

A implementação seguiu uma evolução progressiva, partindo de métodos construtivos básicos até culminar em uma metaheurística refinada.

1. Representação do Grafo: A distância entre os nós (padrão EUC_2D) foi calculada aplicando a função de arredondamento definida pela TSPLIB. Os pesos foram armazenados antecipadamente numa matriz de adjacência na memória. Isso reduz a complexidade da busca por arestas para $\mathcal{O}(1)$, eliminando o recálculo custoso de raízes quadradas nos laços internos de repetição.
2. Construção Inicial (Multi-Start Vizinho Mais Próximo): O processo inicia-se de forma gulosa: a partir do nó atual, transita-se para a cidade vizinha mais próxima ainda não visitada. Para mitigar o viés determinístico dessa escolha inicial, implementou-se o método multi-start, iterando a heurística a partir de múltiplas cidades de origem e conservando a rota de menor custo absoluto.
3. Busca Local:
 - 2-opt: Responsável por "descruzar" arestas da rota atual. O algoritmo inverte a ordem de um subpercurso iterativamente e, caso a perturbação produza redução no custo global, a alteração é mantida. Foi adotada a política de First-Improvement para otimizar o tempo de processamento.
 - Or-opt (Realocação): Após a aplicação do 2-opt, uma rotina adicional tenta remover vértices de forma individual para reinseri-los na melhor posição possível. Esta etapa soluciona micro-ineficiências que a inversão de arestas não consegue detectar.
4. Metaheurística (Iterated Local Search): Métodos de busca local convergem rapidamente para ótimos locais. Para escapar destas regiões e cobrir um espaço maior de soluções, utilizou-se o algoritmo Iterated Local Search (ILS). A técnica perturba o ciclo através do movimento Double-Bridge (rompe a rota em quatro segmentos e os recombina aleatoriamente), e aplica novamente a busca local a partir da nova topologia. Este ciclo é sustentado até o término da janela de tempo imposta.

O pseudocódigo geral do método implementado é apresentado a seguir:

Algorithm 1 Solver ILS para o TSP

```
1:  $ciudades \leftarrow \text{LerEntrada}(\text{stdin})$ 
2:  $M \leftarrow \text{MatrizDistancias}(ciudades)$ 
3:  $s^* \leftarrow \text{MultiStartVMP}(M, t_{vmp})$ 
4:  $s^* \leftarrow \text{BuscaLocal}(s^*, M, t_{bl})$  {2-opt + Or-opt}
5: while  $tempo < t_{max}$  do
6:    $s' \leftarrow \text{DoubleBridge}(s^*)$  {perturbação 4-opt}
7:    $s' \leftarrow \text{BuscaLocal}(s', M, t_{max})$ 
8:   if  $custo(s') < custo(s^*)$  then
9:      $s^* \leftarrow s'$ 
10:  end if
11: end while
12:  $\text{ImprimirTour}(s^*)$ 
```

Onde *BuscaLocal* consiste na aplicação alternada de 2-opt (First-Improvement) e Or-opt (relocação de vértices individuais) até que nenhuma melhoria seja encontrada.

3 Resultados Experimentais

Foram realizados testes automatizados (via `testa_instancias.py`) utilizando um limite de segurança antes do Timeout. A Tabela 1 expõe as métricas colhidas com a heurística ILS.

Tabela 1: Resultados do algoritmo ILS comparados com os ótimos conhecidos

Instância	Dimensão	Ótimo	Nosso Custo	Gap (%)	Timeout (s)
tsp29	29	27603	27603	0.00	15
tsp38	38	6656	6656	0.00	15
tsp51	51	426	426	0.00	15
tsp100	100	20749	20749	0.00	15
tsp150	150	6528	6554	0.40	15
tsp436	436	1482	1492	0.67	30

O algoritmo atingiu o custo ótimo exato em todas as instâncias com até 100 nós (gap de 0%). Para instâncias maiores (150 e 436 nós), os resultados ficaram muito próximos do ótimo, com gaps inferiores a 1%. Observa-se que o tempo de convergência cresce com a dimensão do problema, o que é esperado dado o aumento quadrático do espaço de vizinhança.

Em relação ao comportamento com o aumento da escala, nota-se que o ILS consegue explorar o espaço de soluções de forma eficiente em instâncias de até 100 cidades, convergindo para o ótimo em poucos segundos. A partir de 150 cidades, a busca local demora mais para convergir a cada iteração, reduzindo o número de ciclos de perturbação que o ILS consegue executar dentro do tempo.

A evolução do custo entre a solução inicial puramente construtiva e a metaheurística refinada pode ser observada visualmente. A Figura 1 e a Figura 2 ilustram a eliminação de cruzamentos na rota final em instâncias de diferentes tamanhos.

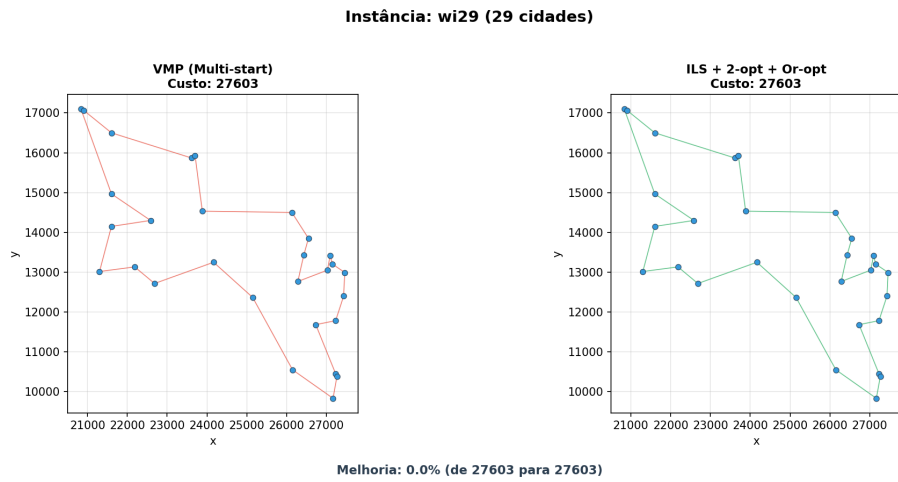


Figura 1: Comparativo entre VMP e ILS na instância tsp29

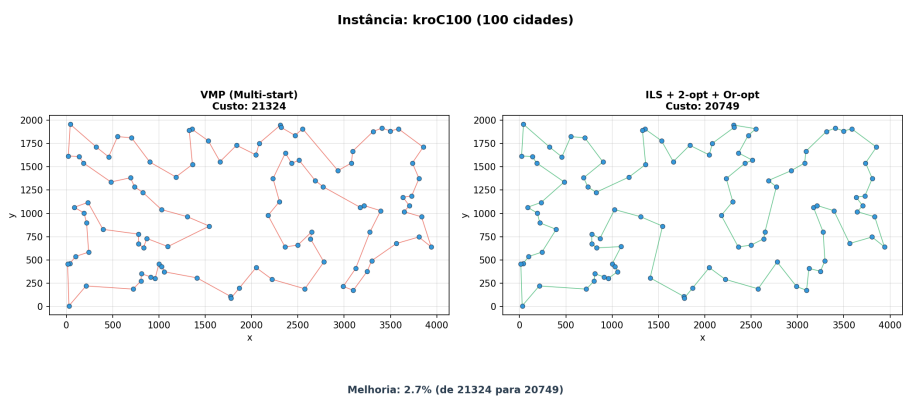


Figura 2: Eliminação drástica de cruzamentos pelo ILS na instância tsp100

4 Uso de Ferramentas de IA

Conforme exigido pelas regras da disciplina, declaro que as ferramentas de IA (ChatGPT e Claude) foram utilizadas apenas como material de apoio, não gerando a lógica nem o código do solver principal. As finalidades foram:

- Dúvidas sobre a linguagem: Uso da IA para entender como o método `.pop()` altera os índices da lista na implementação do Or-opt e a forma correta de copiar listas sem copiar a referência de memória.
- Scripts auxiliares: Auxílio para estruturar o script `testa_instancias.py` (que roda as instâncias em lote via `subprocess`) e na sintaxe da biblioteca `matplotlib` para os gráficos.
- Revisão de texto: Correção de erros de digitação, pontuação e gramática na escrita deste relatório.

5 Conclusão

O desenvolvimento deste projeto evidenciou o comportamento dos algoritmos frente a um problema NP-Difícil. O método construtivo do Vizinheiro Mais Próximo gerou soluções em frações de segundo, mas com um custo total elevado. A aplicação das buscas locais (2-opt e Or-opt) melhorou muito a qualidade das rotas, mas apresentou a tendência de convergir rapidamente para ótimos locais. A adoção da metaheurística ILS, combinada com a perturbação Double-Bridge, foi essencial para contornar esse problema e atingir o ótimo global em instâncias de até 100 nós. O principal desafio superado foi o balanceamento entre tempo de execução e exploração do espaço de busca, garantindo que o programa finalizasse dentro do tempo regulamentar estabelecido.

6 Referências Bibliográficas

- SZWARCFITER, Jayme Luiz. Teoria Computacional de Grafos. Elsevier, 2018.
- LOURENÇO, Helena R.; MARTIN, Olivier C.; STÜTZLE, Thomas. Iterated Local Search. In: Handbook of Metaheuristics. Springer, 2003.
- REINELT, G. TSPLIB - A Traveling Salesman Problem Library. ORSA Journal on Computing, 1991.